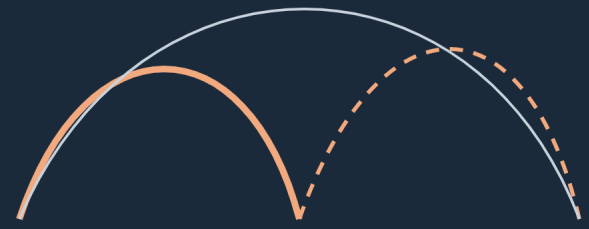
AI 활용 · 자연어 처리

AI는 문장을 어떻게 읽을까

트랜스포머의 원리와 Olist 리뷰 감성 분석



produto não chegou

“상품이 안 왔어요” — 단어끼리 서로를 참고해 뜻을 정합니다

오늘의 질문

별점 없이, 글만 보고 불만을 알아낼 수 있을까?

1. 딥러닝 복습

신경망은 틀리면서 배운다

→

2. 트랜스포머

단어끼리 관계를 계산해 문맥을 읽는다

→

3. Olist 리뷰로 확인

머신러닝 · 딥러닝 · BERT 비교

사용 데이터: 브라질 쇼핑몰 Olist의 고객 리뷰 37,139건 (포르투갈어, 1~2점 = 부정 · 4~5점 = 긍정)

딥러닝 복습

신경망은 '틀리면서' 배운다

① 예측 → ② 정답과 비교 → ③ 오차(손실) 계산 → **④ 가중치 수정**

처음 가중치는 랜덤 → ①~④를 반복할수록 정답을 잘 맞히는 값으로 바뀜 (전체 데이터를 한 바퀴 = 1 epoch)

ReLU

음수는 0, 양수는 그대로
중간층(은닉층)에서 주로 사용

Sigmoid

0~1 사이 확률로 변환
긍정/부정처럼 둘 중 하나 고를 때

Softmax

여러 선택지의 확률, 합계 = 1
셋 이상 중에서 고를 때

딥러닝 복습

데이터 모양에 따라 신경망도 다르다

CNN

이미지를 작은 창으로 훑으며 특징(선, 모양)을 찾음

사진 · 영상

RNN · LSTM

앞에서부터 한 단어씩 순서대로 읽고 기억함

문장 · 시계열

GAN

만드는 쪽과 가려내는 쪽이 경쟁하며 진짜 같은 데이터 생성

이미지 생성

트랜스포머

모든 단어를 한 번에 보고, 서로의 관계를 계산함

오늘의 주인공 · GPT, BERT

트랜스포머는 번역기에서 시작됐다

번역이 어려운 이유: 단어 ≠ 의미

원문	단어 그대로 번역 (✗)	의미를 살린 번역 (✓)
한국 사람들은 정이 많다.	Koreans are full of love.	Koreans value deep emotional bonds with others.
	‘정’은 단순한 사랑이 아님	사람 사이의 유대 · 의리 · 애착
그 사람은 눈치가 없어.	He has no sense.	He can’t read the room.
	‘상식이 없다’로 오해	사회적 상황을 읽는 감각

→ 기계가 번역을 잘하려면 문맥(앞뒤 상황)을 읽어야 한다

번역기의 진화

규칙에서 문맥으로, 다섯 번의 변화

1단계

규칙 기반

사람이 문법 규칙을 직접 입력

한계: 규칙·사투리·신조어가 끝이 없음

2단계

예시 기반

번역해 둔 예문을 바꿔 끼움

한계: 동음이의어(배·배)에 약함

3단계

통계 기반

대량 데이터에서 확률 높은 단어·구를 고름

한계: 문맥을 보지 못함

4단계

신경망 번역

문장 전체를 숫자 묶음 1개로 압축 후 번역

한계: 긴 문장은 앞부분을 잊음

5단계 · 2017

트랜스포머

모든 단어 사이의 관계를 한 번에 계산

→ 오늘날 번역기·챗GPT의 뿌리

번역기의 진화

문맥을 못 보면 생기는 번역

예시 기반 번역

나는 배를 먹는다.

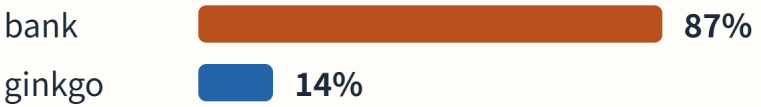
→ I eat a boat. ✗

‘나는 배를 탄다 = I take a boat’ 예문의 ‘배 = boat’를 그대로 가져다 씀

통계 기반 번역 (단어별 확률)

나는 은행 나무에 갔다.

→ I went to the bank tree. ✗



→ 주변 단어(나무)를 함께 봐야 ‘은행’의 뜻이 정해진다 = 어텐션의 출발점

번역기의 진화

신경망 번역의 병목: 문장 전체를 상자 하나에



문제 ① 압축

50단어짜리 긴 문장도 숫자 묶음 1개에 욱여넣음

문제 ② 망각

긴 문장일수록 앞부분과 핵심 단어를 놓침 (LSTM으로 줄였지만 한계)

2017년 · 구글 번역 연구팀

트랜스포머 등장

논문 제목: “Attention Is All You Need” — 필요한 건 어텐션뿐

01

한 번에 읽기

단어를 순서대로가 아니라 동시에 처리 → GPU로 빠르게 학습

02

관계 계산

Self-Attention: 모든 단어가 서로 얼마나 관련 있는지 계산

03

순서 표시

동시에 읽으니 순서를 잃음 → 단어마다 ‘몇 번째’ 정보를 더함

→ BERT, GPT, T5 등 오늘날 거대 언어 모델(LLM)의 공통 뿌리

트랜스포머 들여다보기

문장이 처리되는 다섯 단계

Olist 노트북에서 직접 해 본 단계

이 블록을 6번 반복

① **토큰화**
문장을 조각내고 번호 붙이기

② **임베딩**
번호를 의미가 담긴 숫자 묶음으로

③ **위치 정보**
‘몇 번째 단어인지’ 더하기

④ **어텐션**
단어끼리 관계를 계산해 뜻을 조정

⑤ **정리(FFN)**
단어별 의미를 다듬어 저장

→ 결과: 번역 · 긍정/부정 분류 · 다음 단어 생성

① 토큰화

컴퓨터는 글자가 아니라 번호를 읽는다

produto → 2 **não** → 8 **prazo** → 9

produto = 상품 · não = 아니다 · prazo = 기한(배송 기한)

용어	뜻
단어집	자주 나오는 단어 1만 개에 번호 부여
<OOV> (1번)	단어집에 없는 '모르는 단어'
패딩	길이를 50칸으로 맞추고 빈칸은 0

07_2_sentiment_dl.ipynb

```
[8] tokenizer.texts_to_sequences(X_train)
```

```
원본 문장: Não veio com tudo que disse que vinha
, nas informações do produto ...
숫자 변환: [8, 29, 15, 35, 11, 972, 11, 1922,
217, 475, 6, 2, ...]
```

② 임베딩

번호를 '의미 지도' 위의 좌표로

One-hot



개념도: 실제로는 64개 숫자(64차원) 공간

원-핫 인코딩

1만 칸 중 한 칸만 1 → 모든 단어 사이 거리가 똑같음

임베딩

숫자 64개로 표현 → 비슷한 단어끼리 가까이 모임

Olist 모델: Embedding(10000, 64)
= 단어 1만 개 × 64칸 = 64만 개의 숫자를 학습

④ Self-Attention

‘은행’은 어떤 단어를 보고 뜻을 정할까?



‘나무에’에 가장 집중 → 은행 = ginkgo 🌳



‘돈을’에 가장 집중 → 은행 = bank 🏦

막대 높이 = ‘은행’이 각 단어에 주는 관심(가중치), 이해를 위한 예시

같은 단어도 주변 단어에 따라 다른 숫자(의미)로 바뀐다 — 모든 단어가 동시에 이 계산을 함

④ Self-Attention

어텐션의 재료 Q · K · V = 도서관 검색

Q · Query

검색어

“나는 지금 무엇을 찾고 있지?”
기준이 되는 단어의 질문

K · Key

책 제목표

“나는 이런 단어야”
각 단어의 이름표

V · Value

책 내용

“내가 가진 실제 의미”
가져다 쓸 알맹이 정보

Q와 K 비교 → **관련도 점수** → **비율로 변환 (합 = 1)** → **비율대로 V 섞기**

결과 = 문맥이 반영된 새 단어 벡터 · 비율 변환에 앞에서 본 Softmax를 사용

④ ⑤ 블록 쌓기

여러 관점으로 보고, 정리하고, 반복한다

멀티헤드 어텐션
여러 관점(문법, 의미, 가리키는 대상)으로 어텐션을 동시에 계산

FFN (정리 단계)
단어별로 의미를 4배로 넓혔다가 다시 다듬음 — 지식 저장소 역할

더하기 & 정돈 (Add & Norm)
원래 정보를 더해 잃지 않게 하고, 숫자 크기를 고르게 맞춤



→ 한 층을 지날 때마다 단어의 의미가 조금씩 더 정확해진다 (원조 트랜스포머 = 6층)

인코더와 디코더

BERT는 읽고, GPT는 쓴다

인코더 계열

BERT

상품이 ⇄ 아직 ⇄ 안 왔어요

문장 전체를 앞뒤로 한꺼번에 보고 **이해**

잘하는 일: 분류 · 감성 분석 · 검색

디코더 계열

GPT

상품이 → 아직 → ???

앞 단어만 보고(뒤는 가림) 다음 단어를 **생성**

잘하는 일: 글쓰기 · 대화 · 요약

Olist 감성 분석 ③ 노트북: 다국어 BERT 계열 모델(DistilBERT)을 Hugging Face에서 불러와 사용

직접 만들어 보기 · 도구

TensorFlow는 엔진, Keras는 운전대

TensorFlow

구글이 만든 딥러닝 계산 엔진. 강력하지만 직접 다루기 어려움

Keras

TensorFlow 위에서 동작. 층을 쌓듯 몇 줄로 모델 완성

텐서(Tensor) = 숫자를 담는 상자, 차원 = 상자의 방향 수

0차원 · 숫자 하나

`0.993`

긍정 확률

1차원 · 한 줄

`[8, 29, 15 …]`

리뷰 1개

2차원 · 표

`(23768, 50)`

학습 리뷰 전체

3차원 · 표 여러 장

`(가로, 세로, 3)`

컬러 사진 (RGB)

직접 만들어 보기 · Keras 5단계

모델 만들기 = 시험 공부 과정

① Define
층 쌓기
= 교재 구성

② Compile
채점 기준·공부법
= 손실·옵티마이저

③ Fit
문제집 반복
= 학습

④ Evaluate
최종 시험
= 성능 확인

⑤ Predict
새 리뷰 판정
= 실전

07_2_sentiment_dl.ipynb · 셀 10~11

```
[10] model = Sequential([
         Embedding(input_dim=10000, output_dim=64), # ② 임베딩
         GlobalAveragePooling1D(), # 단어 벡터 평균
         Dense(64, activation="relu"),
         Dense(1, activation="sigmoid"), # 긍정 확률 0~1
     ])
     model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
```

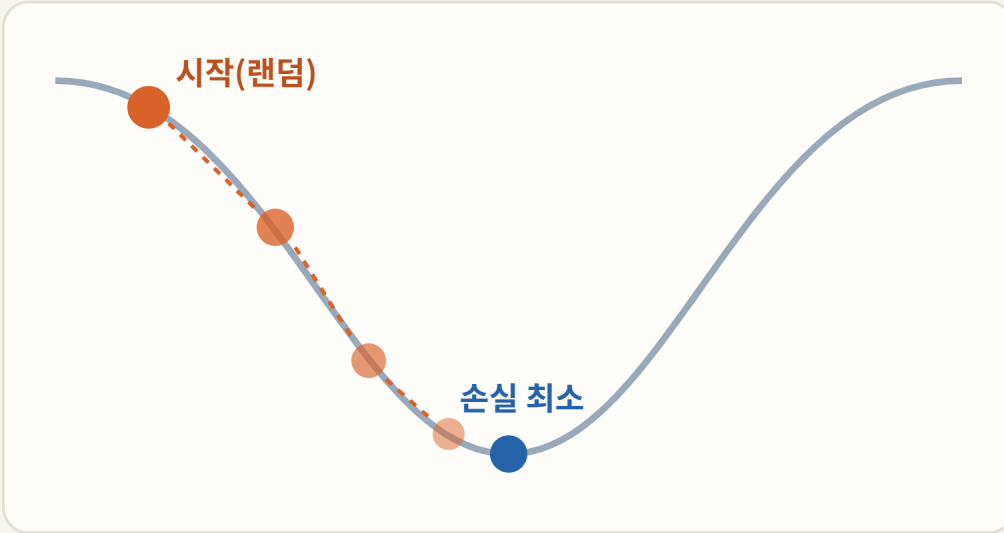
② Compile의 두 재료

얼마나 틀렸나(손실) → 어떻게 고치나(최적화)

손실 함수 = 틀린 정도를 숫자로

종류	계산 · 예시 (예측 5, 정답 10)
MSE	차이의 제곱 평균 → 25
MAE	차이의 절댓값 평균 → 5
교차 엔트로피	분류용. 확신하고 틀릴수록 크게 벌점

MSE·MAE = 숫자 맞히기(회귀) · Olist 긍정/부정 = binary_crossentropy



경사하강법: 기울기를 보고 낮은 쪽으로 한 걸음씩
학습률 = 보폭 (너무 크면 튕겨 나감) · **adam** = 보폭 자동 조절
배치 = 64문장마다 한 번 수정

③ Fit 준비

리뷰 37,139건을 셋으로 나눈다

학습 train 23,768
공부할 문제집

검증 5,943
중간시험

테스트 7,428
최종시험

테스트와 검증 데이터는 처음에 떼어 두고 절대 손대지 않는다 → 그래야 '처음 보는 리뷰'에 대한 실력을 잴 수 있음

긍정이 부정보다 약 2.5배 많다

긍정 26,380 · 부정 10,759 → 그냥 두면 모델이 '긍정'만 외치기 쉬움

class_weight로 보정

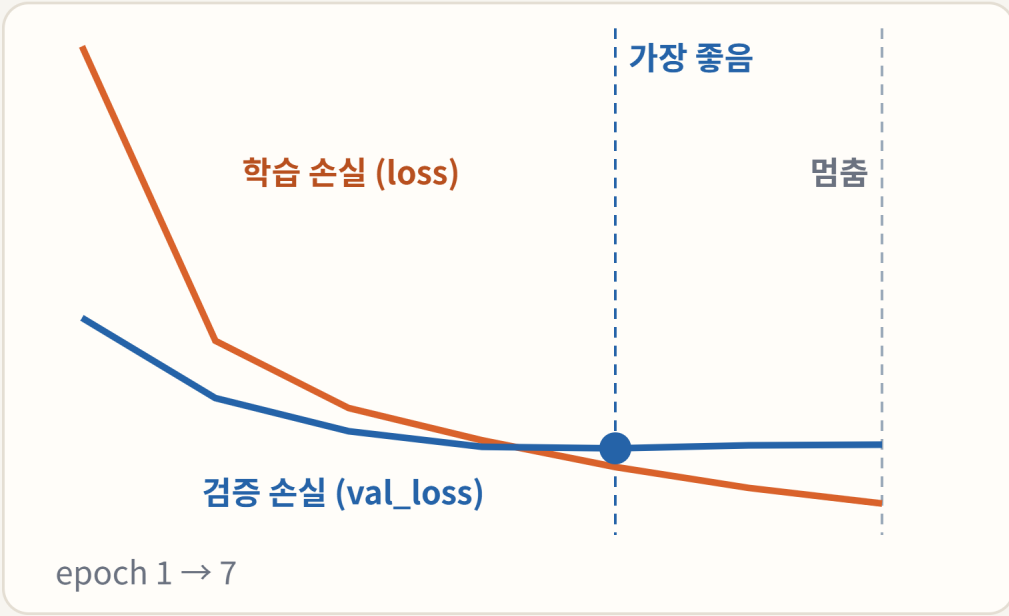
부정 1.73 · 긍정 0.70 → 부정 리뷰를 틀리면 약 2.5배 큰 벌점

③ Fit 화면 읽기

언제 멈출까? 중간시험 점수가 알려준다

07_2_sentiment_dl.ipynb · 셀 12

```
Epoch 1/10 … loss 0.4580 · val_loss 0.2771
Epoch 2/10 … loss 0.2617 · val_loss 0.2236
Epoch 3/10 … loss 0.2172 · val_loss 0.2015
Epoch 4/10 … loss 0.1957 · val_loss 0.1911
Epoch 5/10 … loss 0.1779 · val_loss 0.1901
Epoch 6/10 … loss 0.1639 · val_loss 0.1921
Epoch 7/10 … loss 0.1532 · val_loss 0.1924
가장 좋았던 epoch : 5
```



EarlyStopping: 검증 손실이 2번 연속 나아지지 않으면 멈추고, 가장 좋았던 epoch 5로 되돌림 → 문제집을 외우는 과적합 방지

④ Evaluate · 같은 시험지로 비교

결과: 딥러닝이 항상 이기지는 않는다



정확도 기준 · 직접 학습한 세 모델은 모두 0.92 안팎으로 거의 같음

* BERT는 같은 test 중 500건으로 측정, '중립' 답변 7% 포함 (같은 500건에서 머신러닝 0.918)

결과 해석

왜 이런 결과가 나왔을까?

8단어

리뷰가 짧다

리뷰 길이 중앙값 8단어 → 순서를 읽어도 얻을 정보가 적음

bom

핵심 단어로 판가름

“bom(좋다)”, “não recebi(못 받았다)” 같은 단어만 봐도 대부분 맞힘

540MB

BERT는 남의 데이터로 학습

일반 문장으로 배운 모델 → 우리 데이터로 추가 학습(파인튜닝)하면 오를 수 있음

교훈: 단순한 모델로 기준선부터 → 딥러닝은 ‘확실히’ 나을 때, 비용(시간·용량)까지 따져서

⑤ Predict · 주의할 점

배운 언어 밖의 문장은? 확률을 믿으면 안 된다

07_2_sentiment_dl.ipynb · 셀 14

```
predict_sentiment("produto muito bom gostei")
predict_sentiment("produto ruim nao gostei")
predict_sentiment("배송이 빠르고 상품이 좋아요")
```

```
('긍정', 0.993) # 좋은 제품이에요
('부정', 0.083) # 나쁜 제품이에요
한국어 숫자 변환: [[1, 1, 1, 1]]
('긍정', 0.834)
```

한국어 4단어 → 전부 1번

1번 = <OOV>(모르는 단어). '모르는 단어 4개'만 보고 낸 답이라 0.834라도 의미 없음

BERT는 다국어 단어집 내장

한국어도 입력 가능. 대신 용량이 크고(약 540MB) Olist 리뷰로 배우지 않음

정리

꼭 가져갈 세 가지

1 **트랜스포머** = 모든 단어 사이의 관계(어텐션)를 한 번에 계산해 문맥을 읽는 구조 → BERT·GPT의 뿌리

2 **컴퓨터는 숫자로 읽는다**: 문장 → 토큰(번호) → 임베딩(의미 좌표) → 어텐션(문맥 반영)

3 **성능은 같은 시험지로 비교**: 짧은 Olist 리뷰에서는 가벼운 머신러닝(0.917)도 딥러닝(0.924)만큼 강하다